

Generative Adversarial Networks

Yu Kong

Department of Computer Science and Engineering

Michigan State University



Donald J. Trump ✓
@realDonaldTrump

You cannot trust CNN! They are FAKE!!!

RETWEETS

7,771

LIKES

2,094



11:07 AM - 21 Nov 2017



364



8K

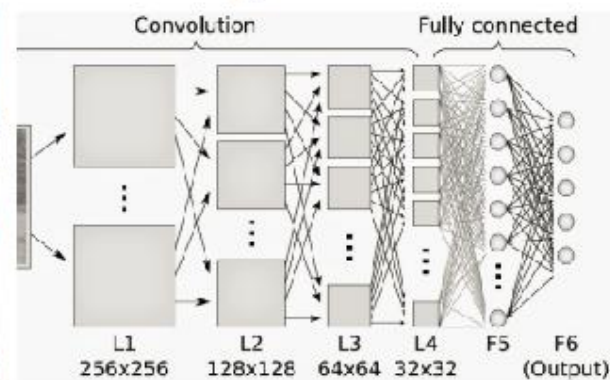


2K

**what people think
he's referring to**



**what he's actually
referring to**

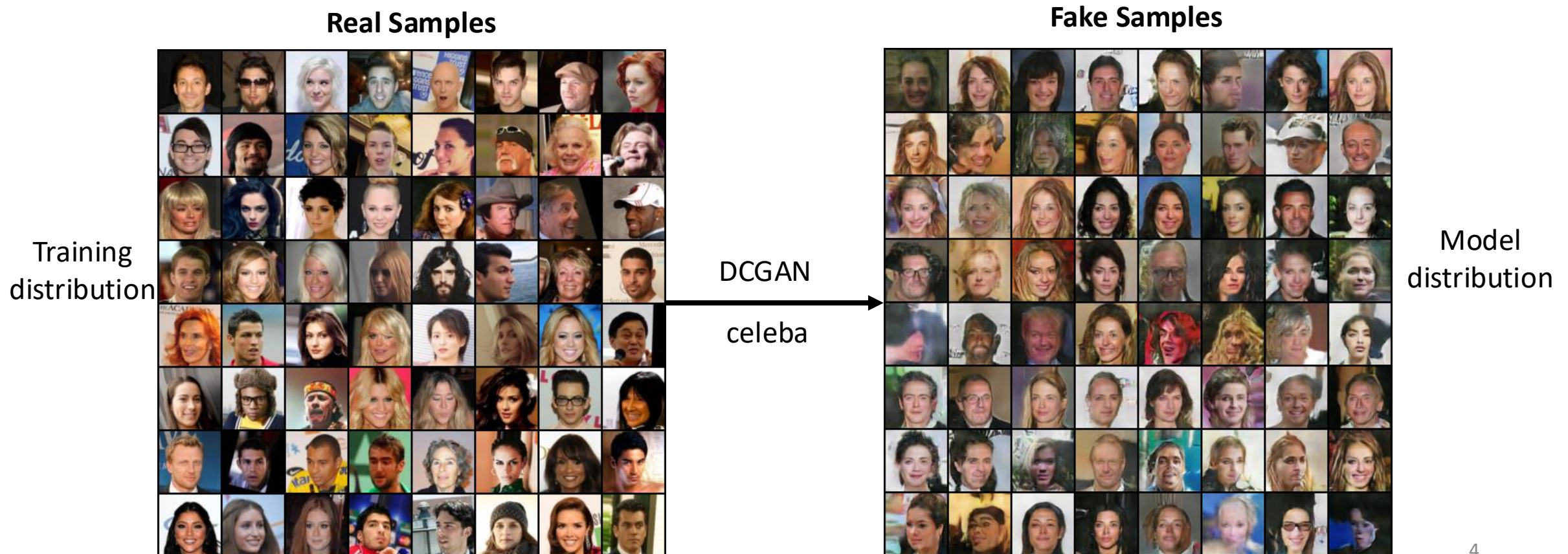


GAN



What Are GANs?

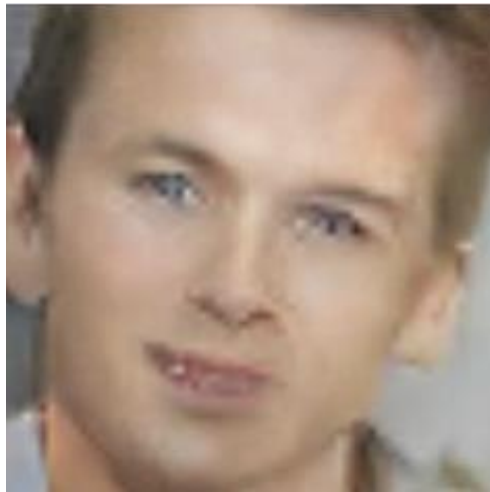
- GANs (invented by Ian Goodfellow in 2014) are a class of DL models that are taught to **capture the training data's distribution so we can generate new data from that same distribution.**



GANs Are Evolving



2014



2015



2016



2017

1024× 1024

NVIDIA, Progressive Growing of GANs

Face Frontalization



Normalized Face

Face Aging



StarGAN

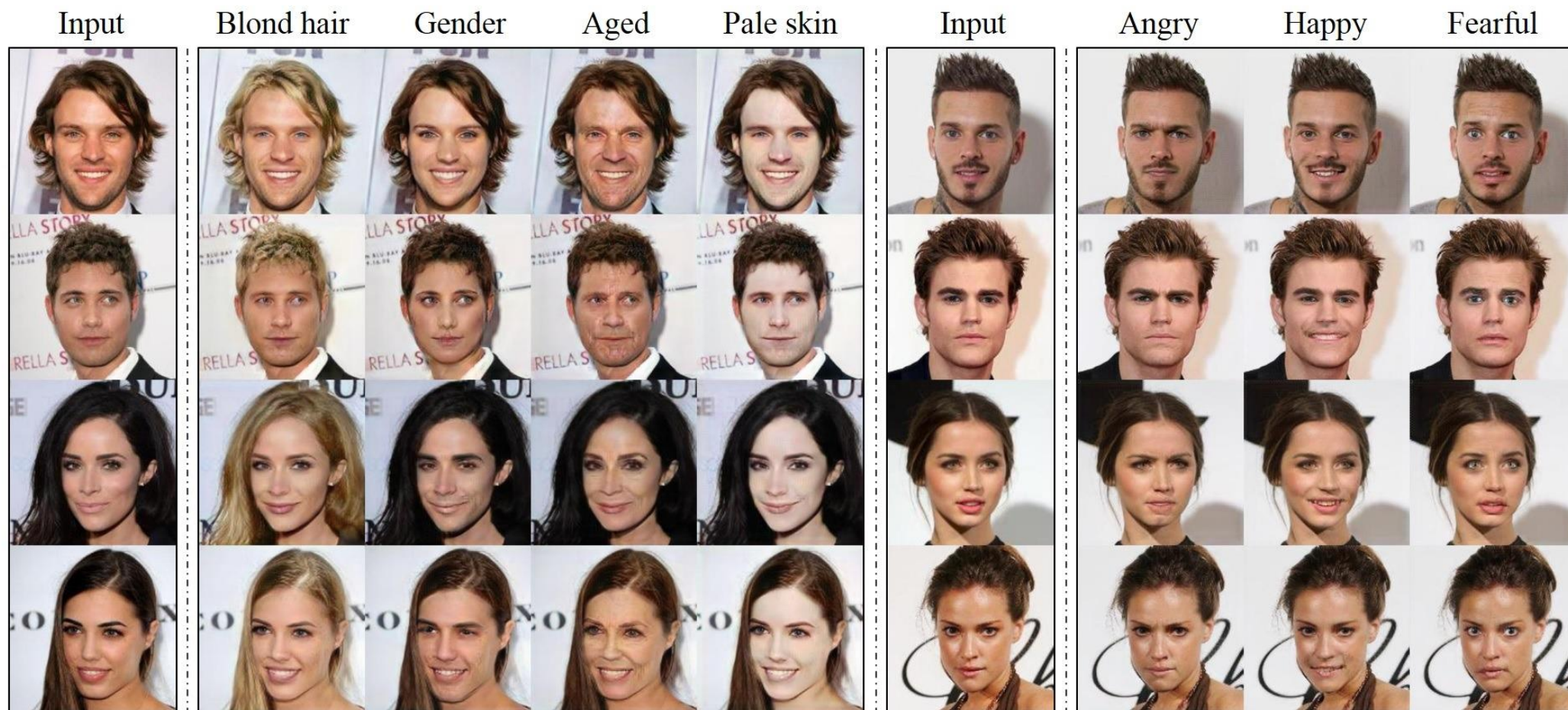
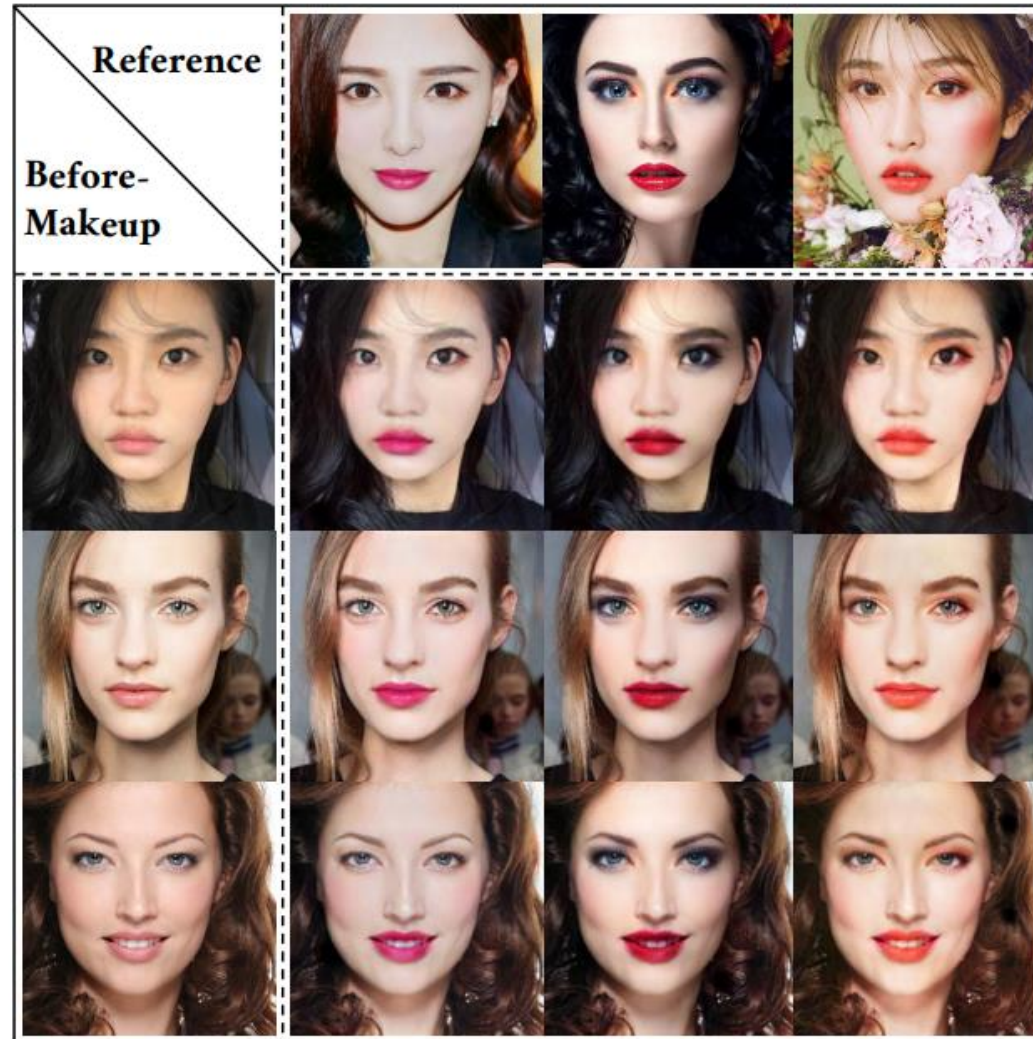


image-to-image translations for multiple domains

BeautyGAN



STGAN

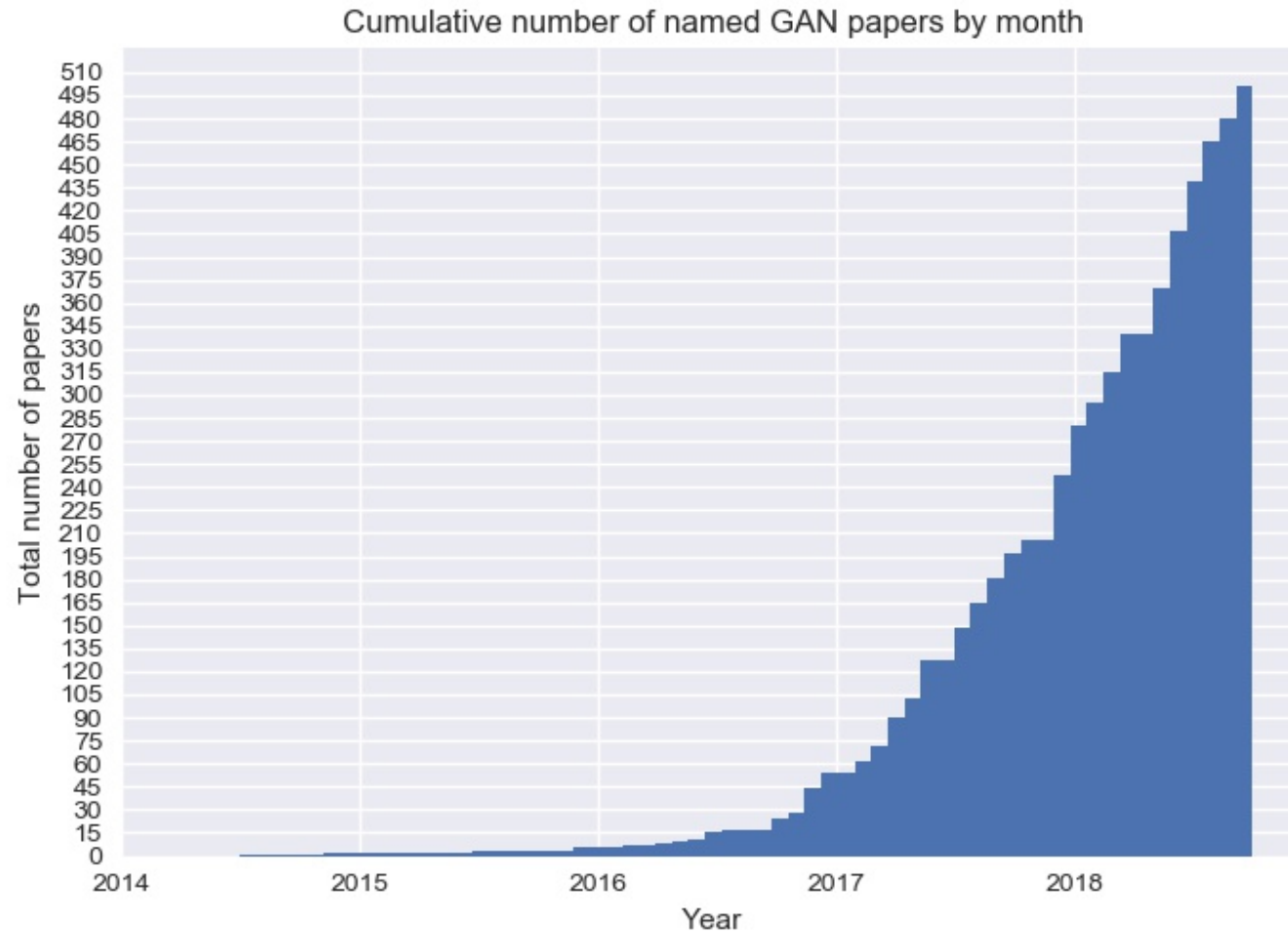


StyleGAN



1024× 1024

The-GAN-Zoo



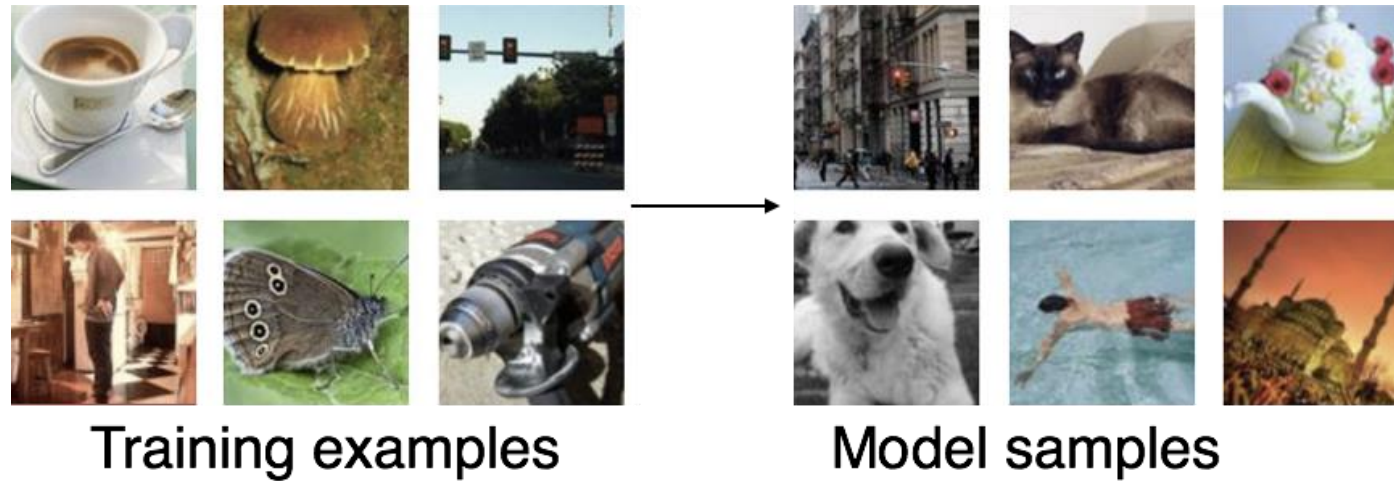
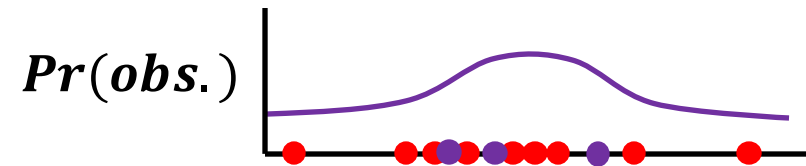
Probabilistic Generative Models



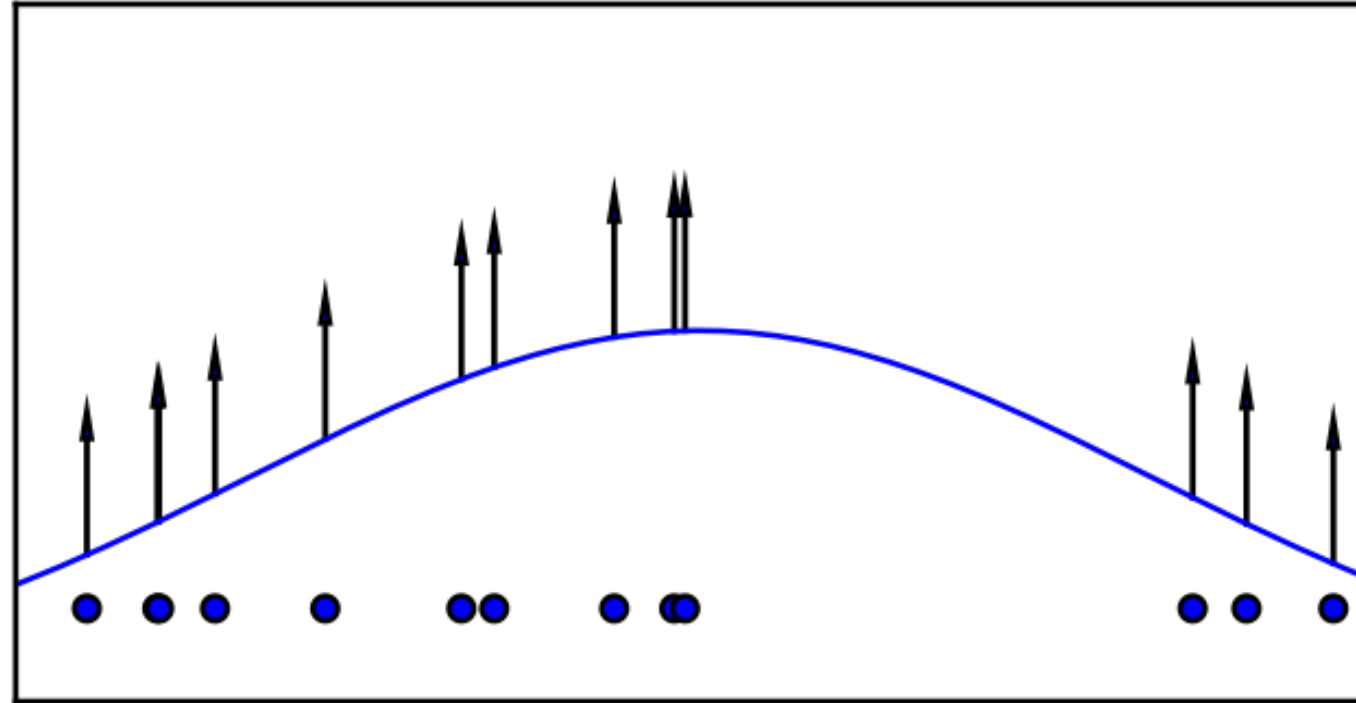
Density Estimation

- $Pr(\text{observation}) = Pr(\text{synthetic obs.})$

Synthesizing Examples From Probabilistic Generative Model



Maximum Likelihood Estimation



$$\theta^* = \arg \max_{\theta} \mathbb{E}_{x \sim p_{\text{data}}} \log p_{\text{model}}(x \mid \theta)$$

Density function $\Pr_{\text{model}}(x|\theta)$

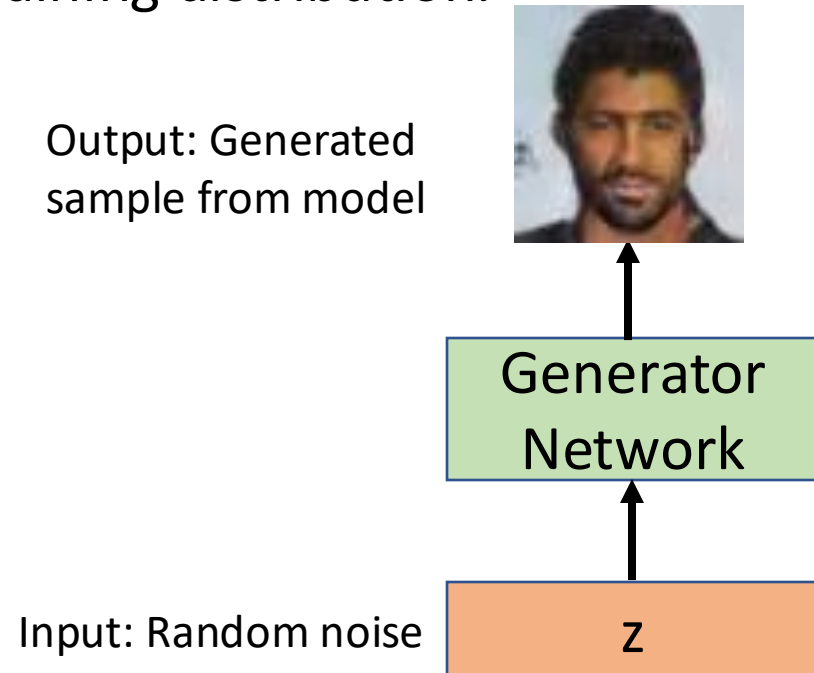
- Explicit and analytical
 - e.g., Gaussian
 - can sample directly from model
- Explicit and approximate
 - e.g., Boltzmann machine
 - can estimate probability by running Markov chain monte carlo
- Implicit
 - GAN
 - can't estimate probability but can draw from distribution with given probability

GAN

- Problem: Difficult to directly sample from complex, high-dimensional training distribution.
- Solution: Sample from a simple distribution instead, e.g., random noise (normal distribution), and then learn transformation to training distribution.

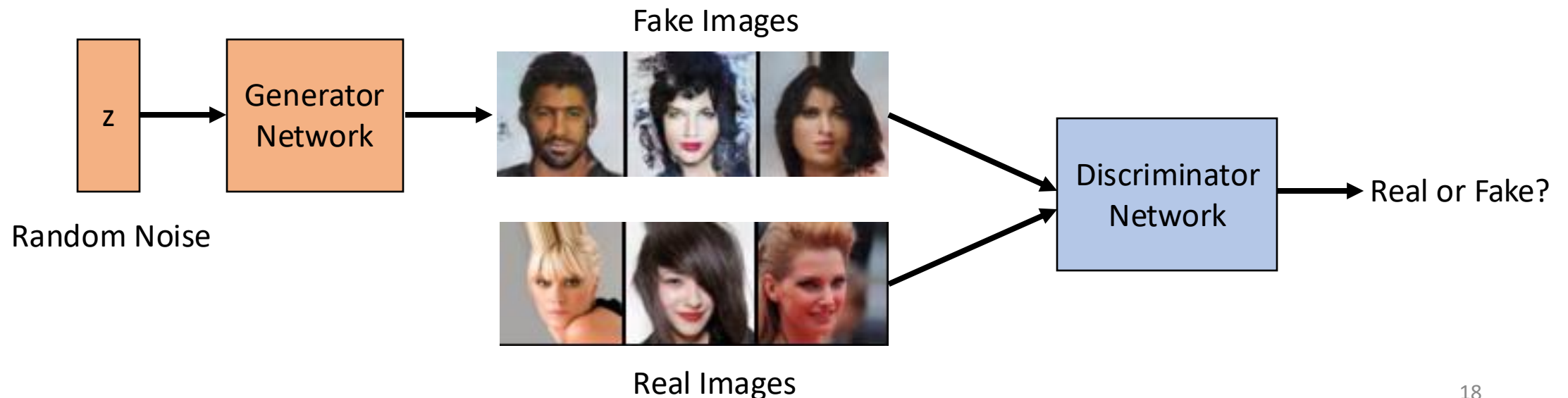
Q: How do we represent this complex transformation?

A: A neural network.



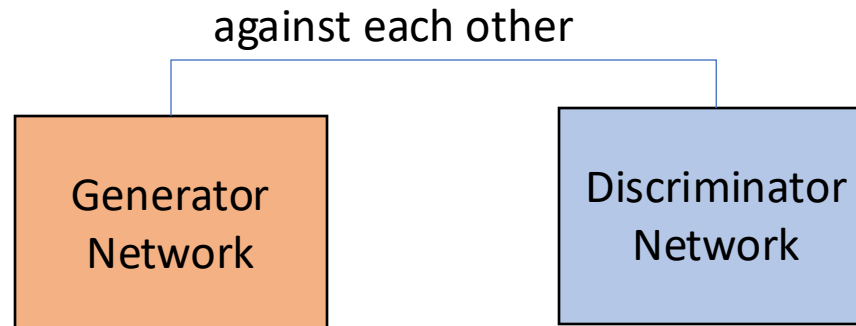
GAN

- **Generator network:** try to generate 'fake' images that resemble the real (training) images.
- **Discriminator network:** try to look at an image and determine whether or not it is a real training image or a fake image from the generator.

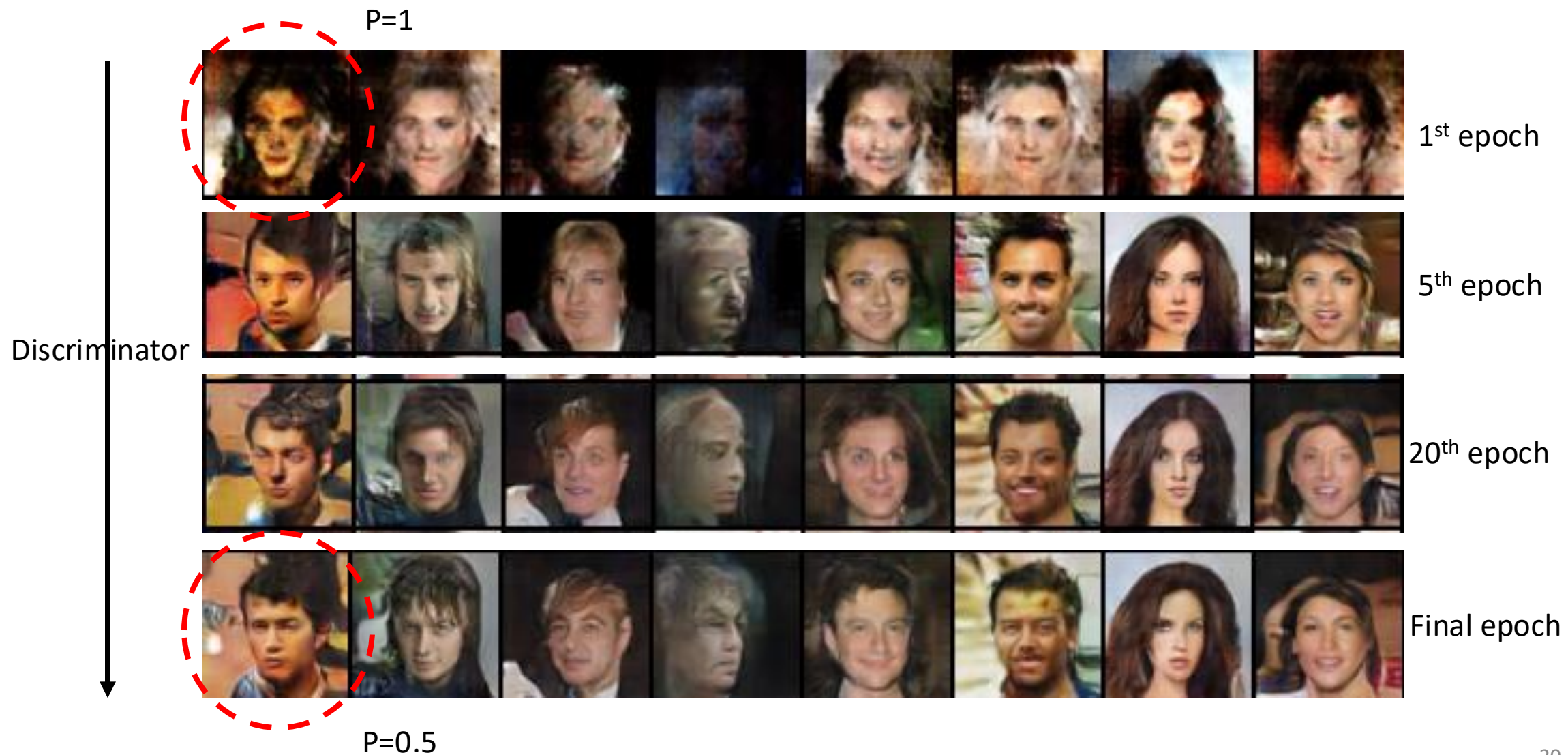


GAN

- During training, the generator is constantly trying to outsmart the discriminator by generating better and better fakes, while the discriminator is working to become a better detective and correctly classify the real and fake images.



GAN



GAN

- **Generator network:** try to generate 'fake' images that resemble the real (training) images.
- **Discriminator network:** try to look at an image and determine whether or not it is a real training image or a fake image from the generator.

minimax objective function:

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{data}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))]$$

Q: What is the idea of this objective function?

- z is a latent vector sampled from a normal distribution.
- $D(x)$ is the discriminator network.
- $G(z)$ denotes the generator network which maps the latent vector z to data-space.

GAN

- **Generator network:** try to generate 'fake' images that resemble the real (training) images.
- **Discriminator network:** try to look at an image and determine whether or not it is a real training image or a fake image from the generator.

minimax objective function:

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{data}(x)} \underbrace{\left[\log D(x) \right]}_{\text{Discriminator output for real data } x} + \mathbb{E}_{z \sim p_z(z)} \underbrace{\left[\log(1 - D(G(z))) \right]}_{\text{Discriminator output for generated fake data } G(z)}$$

The output of $D(x)$ is a probability indicating that being **HIGH** when x comes from training data and **LOW** when x comes from the generator. $D(x)$ can be considered as a **binary classifier**, where the label for real and fake data is 1 and 0, respectively.

GAN

- **Generator network:** try to generate 'fake' images that resemble the real (training) images.
- **Discriminator network:** try to look at an image and determine whether or not it is a real training image or a fake image from the generator.

minimax objective function:

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{data}(x)} \underbrace{\left[\log D(x) \right]}_{\text{Discriminator output for real data } x} + \mathbb{E}_{z \sim p_z(z)} \underbrace{\left[\log(1 - D(G(z))) \right]}_{\text{Discriminator output for generated fake data } G(z)}$$

- D wants to **maximize objective** such that $D(x)$ is close to 1 (real) and $D(G(z))$ is close to 0 (fake).
- G wants to **minimize objective** such that $D(G(z))$ is close to 1.

GAN

- **Generator network:** try to generate 'fake' images that resemble the real (training) images.
- **Discriminator network:** try to look at an image and determine whether or not it is a real training image or a fake image from the generator.

minimax objective function:

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{data}(x)} \underbrace{\left[\log D(x) \right]}_{\text{Discriminator output for real data } x} + \mathbb{E}_{z \sim p_z(z)} \underbrace{\left[\log(1 - D(G(z))) \right]}_{\text{Discriminator output for generated fake data } G(z)}$$

- *The equilibrium reaches when $p_g = p_{data}$, where D guesses randomly.*
- *The convergence of GAN still requires further analysis.*

GAN

- **Generator network:** try to generate 'fake' images that resemble the real (training) images.
- **Discriminator network:** try to look at an image and determine whether or not it is a real training image or a fake image from the generator.

minimax objective function:

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{data}(x)} \underbrace{\left[\log D(x) \right]}_{\substack{\text{Discriminator output} \\ \text{for real data } x}} + \mathbb{E}_{z \sim p_z(z)} \underbrace{\left[\log(1 - D(G(z))) \right]}_{\substack{\text{Discriminator output} \\ \text{for generated fake data } G(z)}}$$

- *In practice: maximize $\log(D(x)) + \log(1 - D(G(z)))$ and maximize $\log(D(G(z)))$*

$$L_D^{GAN} = E[\log(D(x))] + E[\log(1 - D(G(z)))]$$

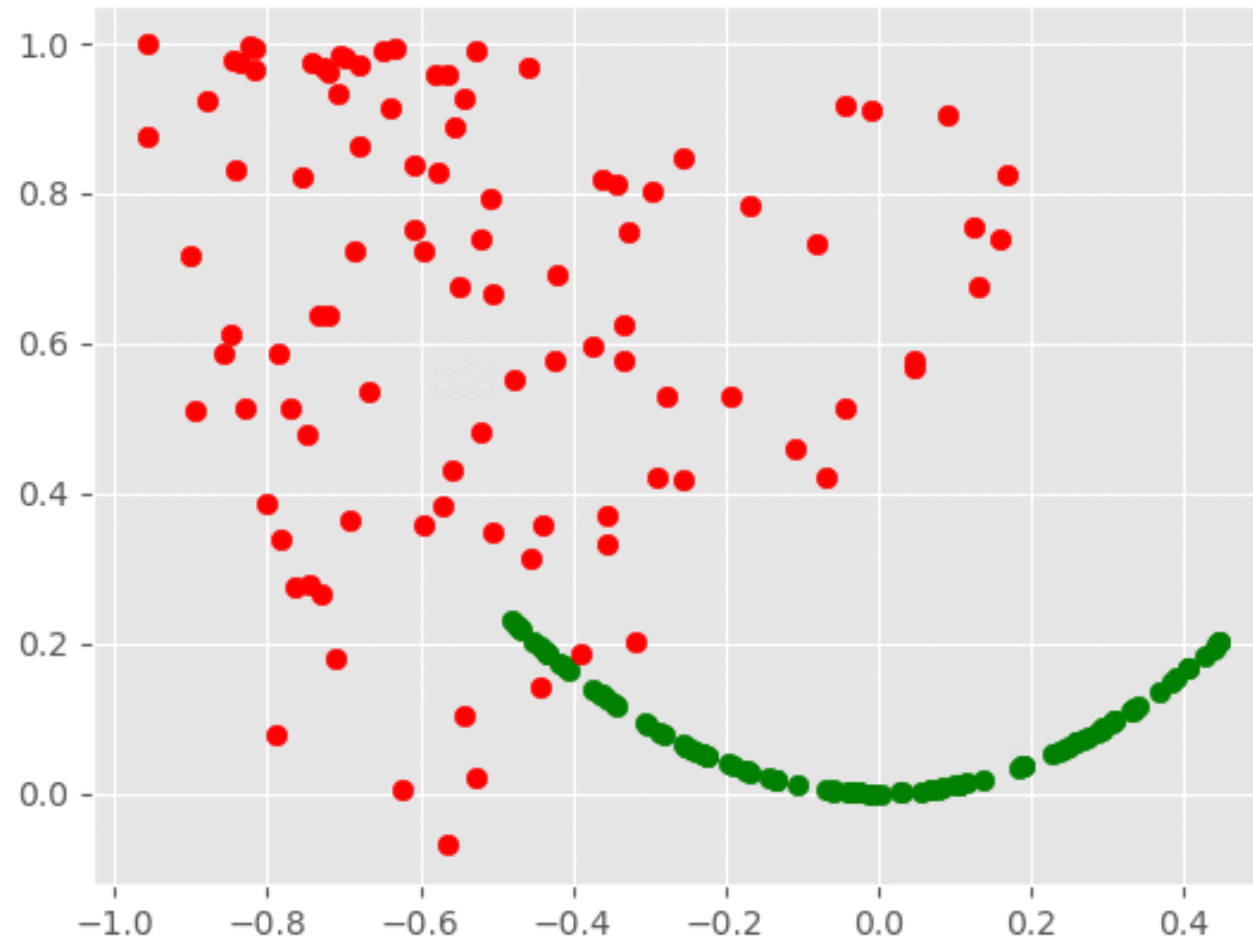
$$L_G^{GAN} = E[\log(D(G(z)))]$$

When will the learning converge?

Equilibrium

- The equilibrium of this game is when the generator is generating perfect fakes that look as if they came directly from the training data, and the discriminator is left to always guess at 50% confidence that the generator output is real or fake.

Distribution learning



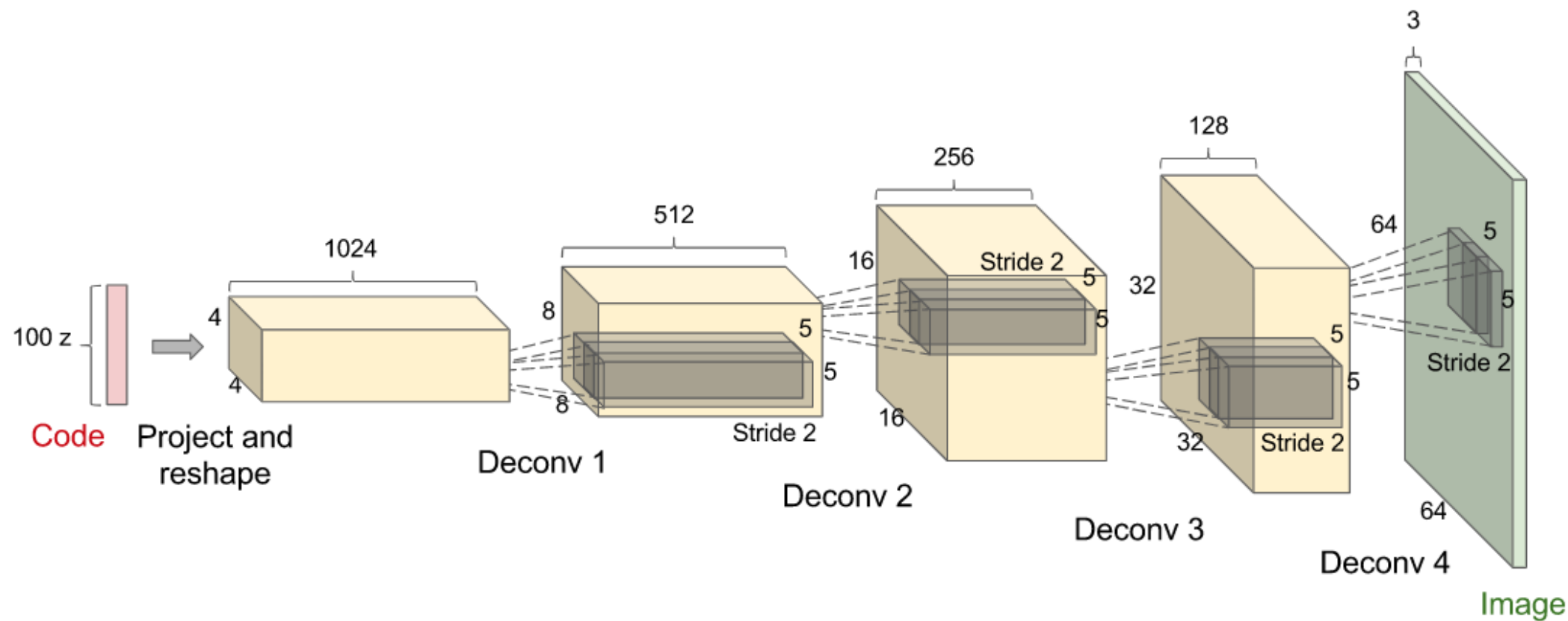
MATLAB tutorial

- <https://www.mathworks.com/help/deeplearning/ug/train-generative-adversarial-network.html>

Deep Convolutional Generative Adversarial Network (DCGAN)

- A DCGAN is a direct extension of the GAN, where it explicitly **uses convolutional and convolutional-transpose layers in the discriminator and generator**, respectively.
- Discriminator: **strided convolution layers**, batch norm layers, and Leaky ReLU activations. **Input is a 3x64x64 input image and output is a scalar probability.**
- Generator: **convolutional-transpose layers**, batch norm layers, and ReLU activations. **The input is a latent vector, z , that is drawn from a standard normal distribution and the output is a 3x64x64 RGB image.**

DCGAN



<https://openai.com/blog/generative-models/>

DCGAN

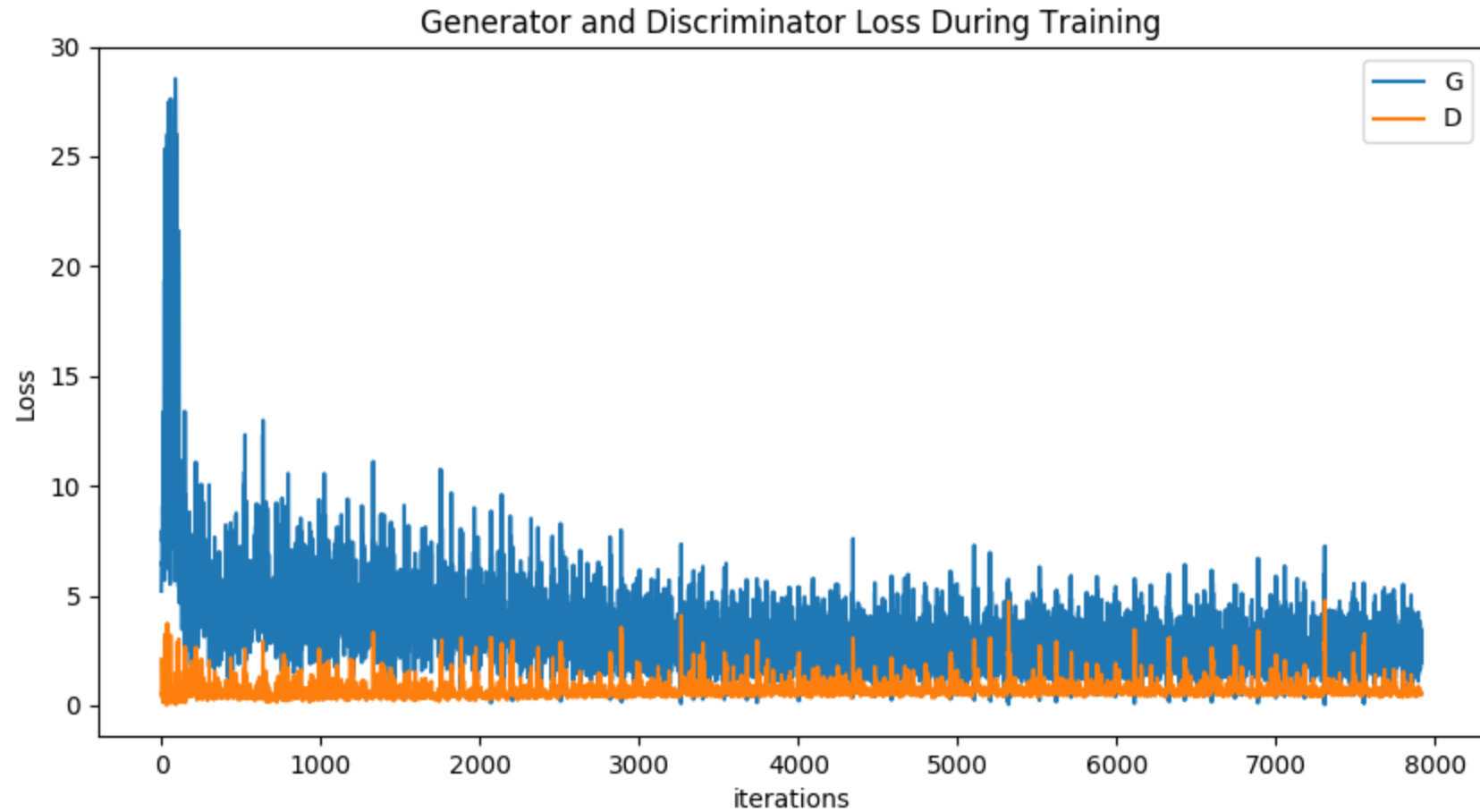
```
class Generator(nn.Module):
    def __init__(self, ngpu):
        super(Generator, self).__init__()
        self.ngpu = ngpu
        self.main = nn.Sequential(
            # input is Z, going into a convolution
            nn.ConvTranspose2d( nz, ngf * 8, 4, 1, 0, bias=False),
            nn.BatchNorm2d(ngf * 8),
            nn.ReLU(True),
            # state size. (ngf*8) x 4 x 4
            nn.ConvTranspose2d(ngf * 8, ngf * 4, 4, 2, 1, bias=False),
            nn.BatchNorm2d(ngf * 4),
            nn.ReLU(True),
            # state size. (ngf*4) x 8 x 8
            nn.ConvTranspose2d( ngf * 4, ngf * 2, 4, 2, 1, bias=False),
            nn.BatchNorm2d(ngf * 2),
            nn.ReLU(True),
            # state size. (ngf*2) x 16 x 16
            nn.ConvTranspose2d( ngf * 2, ngf, 4, 2, 1, bias=False),
            nn.BatchNorm2d(ngf),
            nn.ReLU(True),
            # state size. (ngf) x 32 x 32
            nn.ConvTranspose2d( ngf, nc, 4, 2, 1, bias=False),
            nn.Tanh()
            # state size. (nc) x 64 x 64
        )
```

Generator

```
class Discriminator(nn.Module):
    def __init__(self, ngpu):
        super(Discriminator, self).__init__()
        self.ngpu = ngpu
        self.main = nn.Sequential(
            # input is (nc) x 64 x 64
            nn.Conv2d(nc, ndf, 4, 2, 1, bias=False),
            nn.LeakyReLU(0.2, inplace=True),
            # state size. (ndf) x 32 x 32
            nn.Conv2d(ndf, ndf * 2, 4, 2, 1, bias=False),
            nn.BatchNorm2d(ndf * 2),
            nn.LeakyReLU(0.2, inplace=True),
            # state size. (ndf*2) x 16 x 16
            nn.Conv2d(ndf * 2, ndf * 4, 4, 2, 1, bias=False),
            nn.BatchNorm2d(ndf * 4),
            nn.LeakyReLU(0.2, inplace=True),
            # state size. (ndf*4) x 8 x 8
            nn.Conv2d(ndf * 4, ndf * 8, 4, 2, 1, bias=False),
            nn.BatchNorm2d(ndf * 8),
            nn.LeakyReLU(0.2, inplace=True),
            # state size. (ndf*8) x 4 x 4
            nn.Conv2d(ndf * 8, 1, 4, 1, 0, bias=False),
            nn.Sigmoid()
        )
```

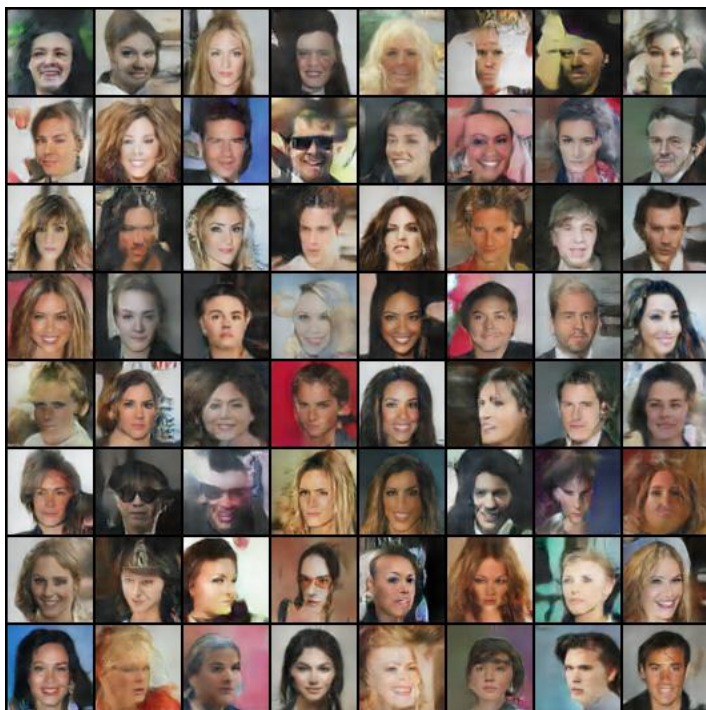
Discriminator

DCGAN



DCGAN

5th epoch



20th epoch



25th epoch



DCGAN



Fake face generation demo

How to Train a Better GAN?

Tricks • Label smoothing • Use instance normalization • Use labels • ...	New objectives • EBGAN • LSGAN • WGAN • BEGAN • fGAN • ...
Surrogate or auxiliary objective • UnrolledGAN • WGAN-GP • DRAGAN • ...	Network architecture • LAPGAN • PGGAN • ...

Least Squares Generative Adversarial Networks (LSGAN)

- Discriminator

GAN $\min_D E_{x \sim p_X} [-\log D(x)] + E_{z \sim p_Z} [-\log(1 - D(G(z)))]$



LSGAN $\min_D E_{x \sim p_X} [(D(x) - 1)^2] + E_{z \sim p_Z} [D(G(z))^2]$

MSE Loss

```
# update D network
self.D_optimizer.zero_grad()

D_real = self.D(x_)
D_real_loss = self.MSE_loss(D_real, self.y_real_)

G_ = self.G(z_)
D_fake = self.D(G_)
D_fake_loss = self.MSE_loss(D_fake, self.y_fake_)

D_loss = D_real_loss + D_fake_loss
self.train_hist['D_loss'].append(D_loss.item())

D_loss.backward()
self.D_optimizer.step()

# update G network
self.G_optimizer.zero_grad()

G_ = self.G(z_)
D_fake = self.D(G_)
G_loss = self.MSE_loss(D_fake, self.y_real_)
self.train_hist['G_loss'].append(G_loss.item())

G_loss.backward()
self.G_optimizer.step()
```

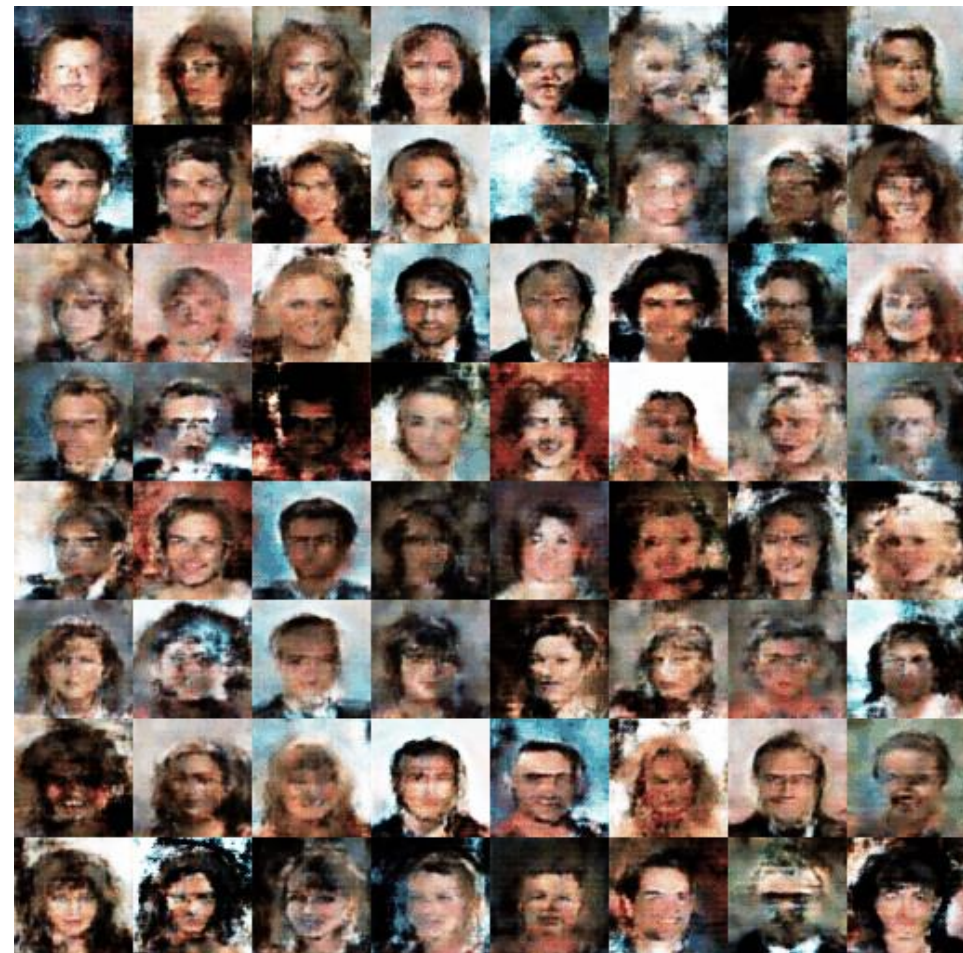
- Generator

GAN $\min_G E_{z \sim p_Z} [-\log D(G(z))]$



LSGAN $\min_G E_{z \sim p_Z} [(D(G(z)) - 1)^2]$

LSGAN



video demo

Wasserstein Generative Adversarial Networks (WGAN)

- Training WGANs does not require maintaining a careful balance in training of the discriminator and the generator.
- Training WGANs does not require a careful design of the network architecture.
- The mode dropping phenomenon that is typical in GANs is also drastically reduced.
- The ability to continuously estimate the EM distance by training the discriminator to optimality with the learning curve visualization for debugging, hyperparameter searches and sample quality observation.

WGAN

- Discriminator

GAN $\max_D E_{x \sim p_X} [\log D(x)] + E_{z \sim p_Z} [\log(1 - D(G(z)))]$



WGAN $\max_D E_{x \sim p_X} [D(x)] - E_{z \sim p_Z} [D(G(z))]$

```
# update D network
self.D_optimizer.zero_grad()

D_real = self.D(x_)
D_real_loss = -torch.mean(D_real)

G_ = self.G(z_)
D_fake = self.D(G_)
D_fake_loss = torch.mean(D_fake)

D_loss = D_real_loss + D_fake_loss

D_loss.backward()
self.D_optimizer.step()

# clipping D
for p in self.D.parameters():
    p.data.clamp_(-self.c, self.c)
```

- Generator

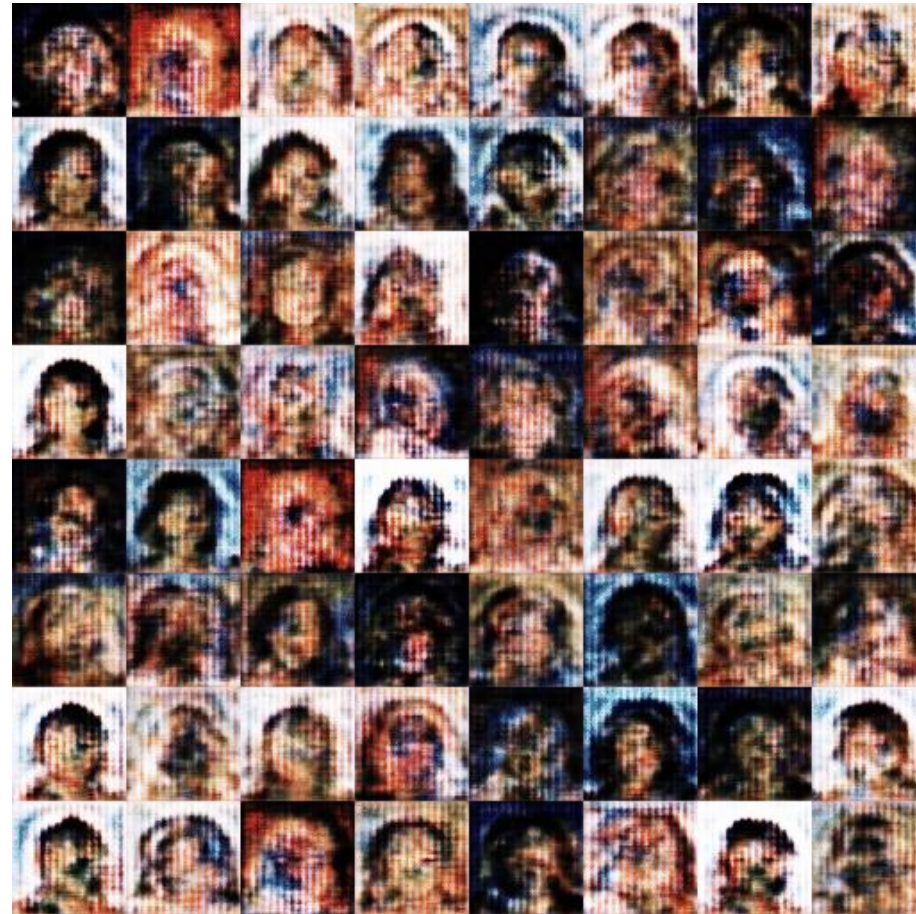
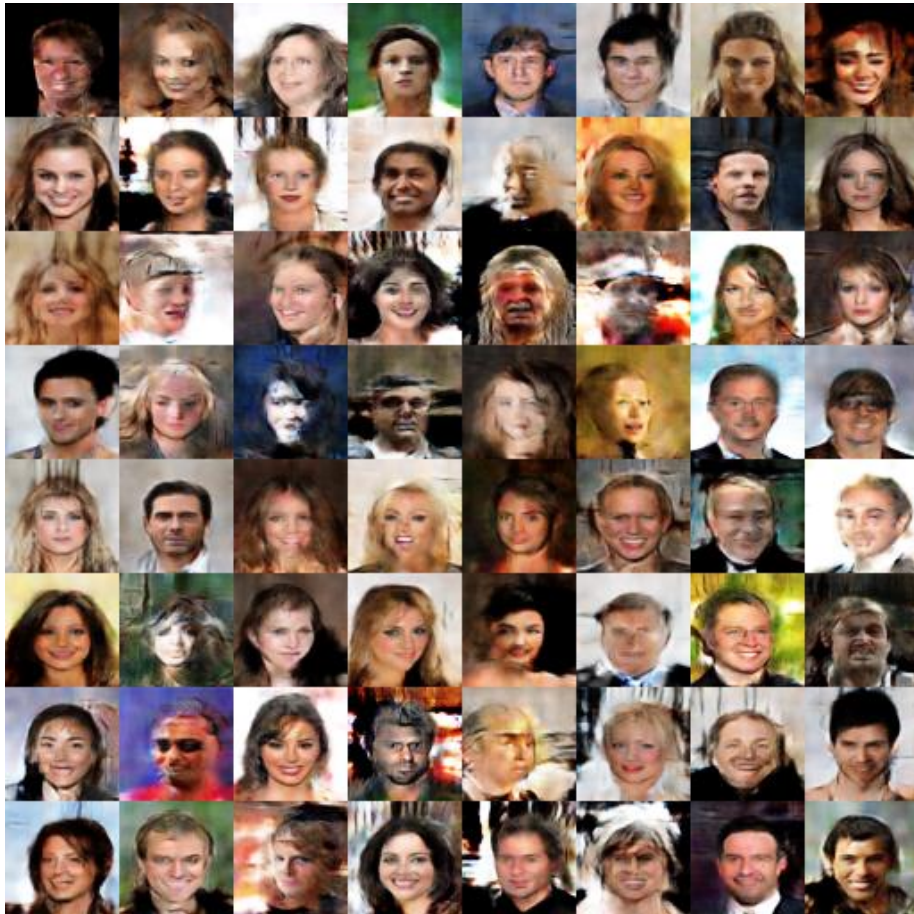
GAN $\max_G E_{z \sim p_Z} [\log D(G(z))]$



WGAN $\max_G E_{z \sim p_Z} [D(G(z))]$

No Sigmoid at the end of D
No Log loss computation
Weight clip for D
Update G network after critic

WGAN



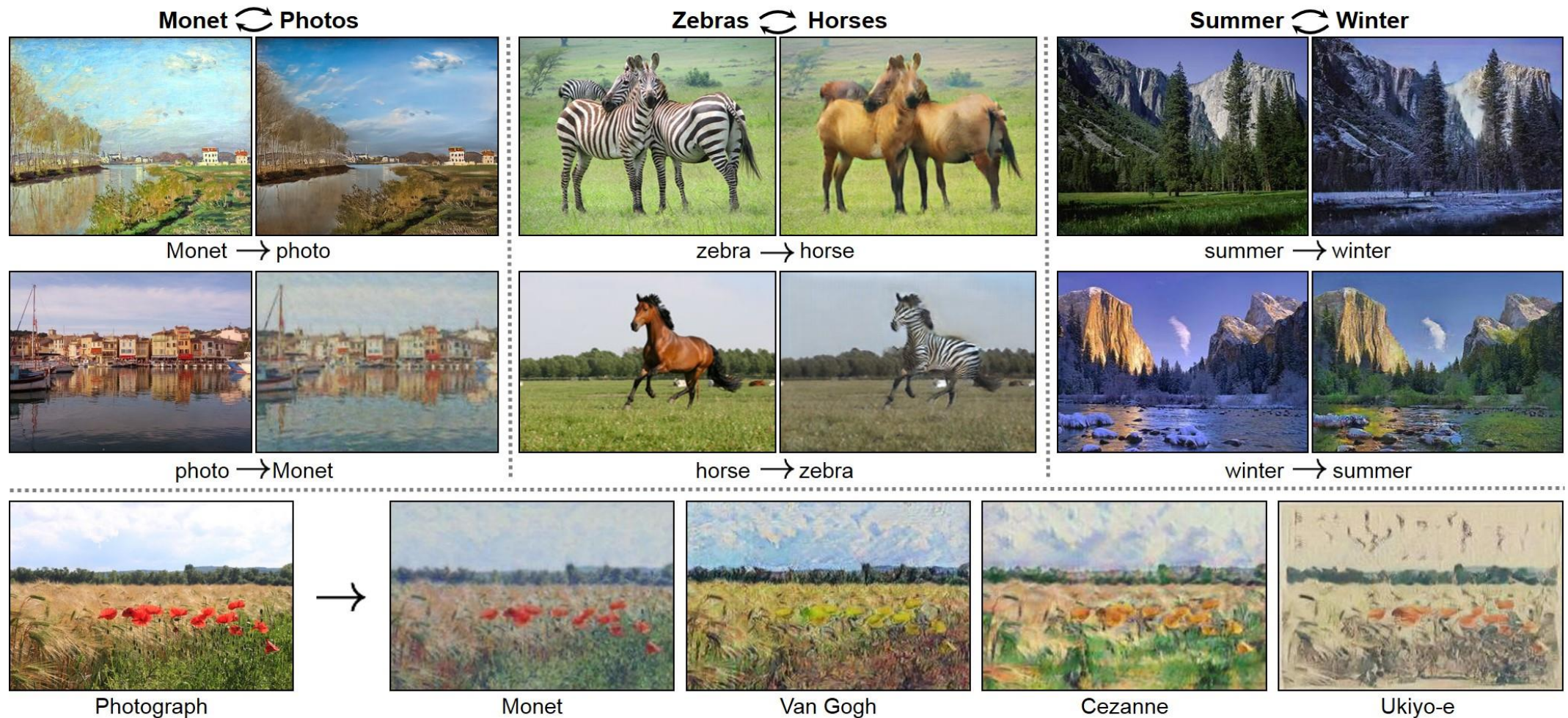
video demo

WGAN-GP

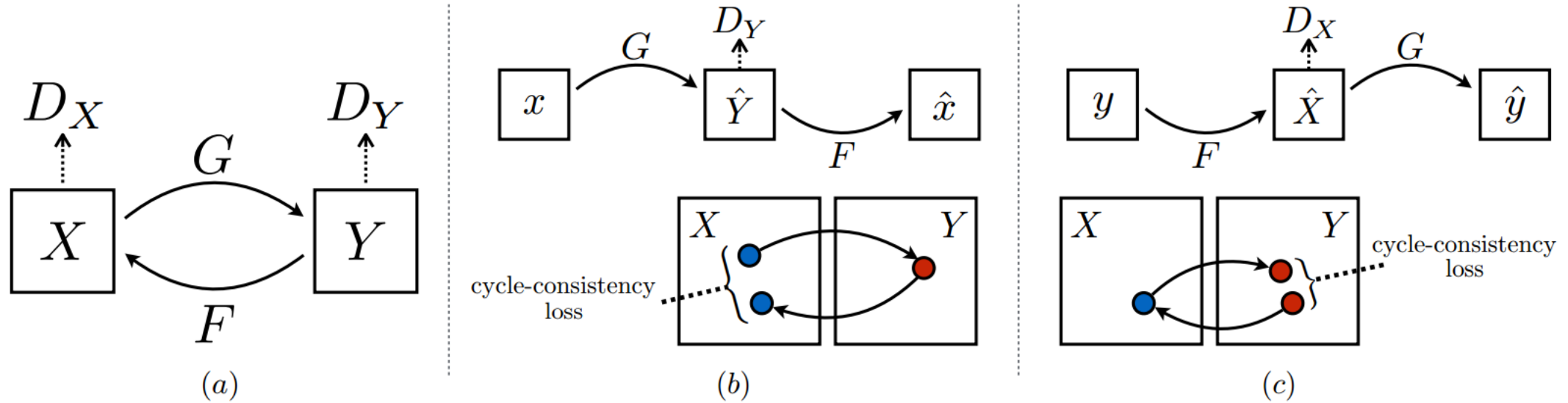


video demo

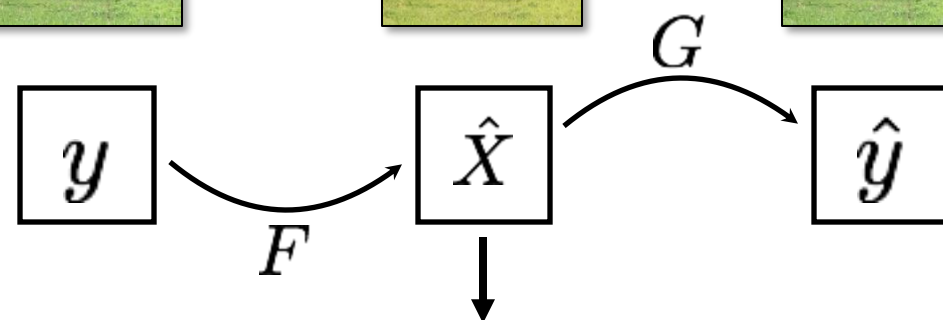
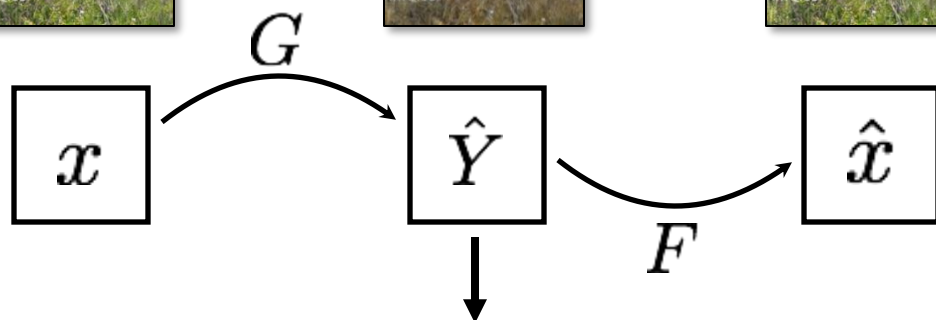
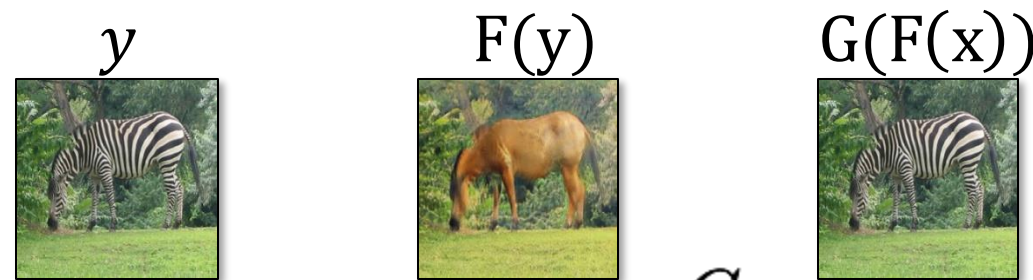
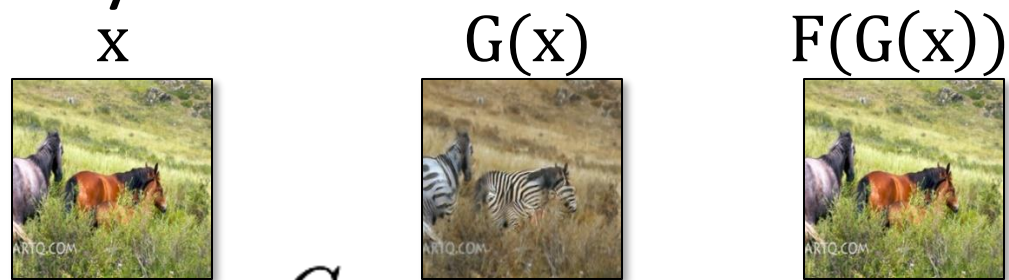
CycleGAN (Image-to-image translation)



CycleGAN



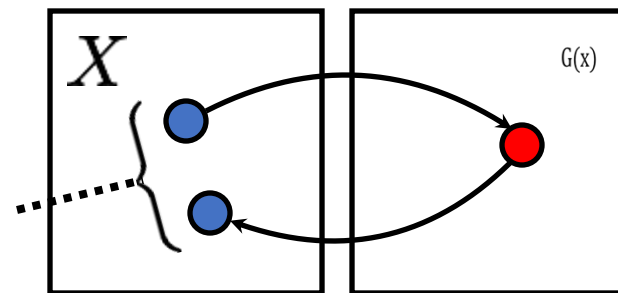
CycleGAN



$D_Y(G(x))$

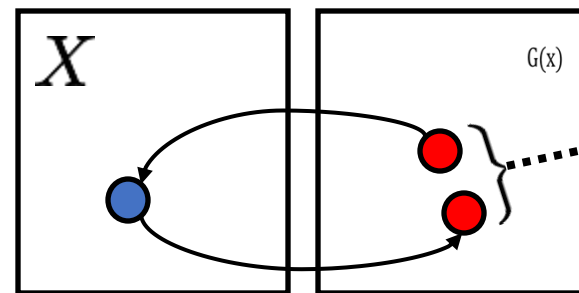
$D_G(F(x))$

Reconstruction
error



$$\|F(G(x)) - x\|_1$$

Reconstruction
error



$$\|G(F(y)) - y\|_1$$